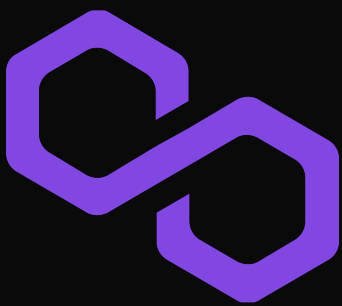




Security Assessment Final Report



Polygon-Trails-Wallet

May 2026

Prepared for Polygon

Table of Contents

Project Summary.....	3
Project Scope.....	3
Project Overview.....	3
Protocol Overview.....	3
Assessment Methodology.....	5
Threat and Security Overview.....	6
Findings Summary.....	8
Severity Matrix.....	8
Detailed Findings.....	9
Low Severity Issues.....	10
L-01: TimedRefundSapient enforces only .transfer calls, which may silently ignore failed transfers for tokens that don't revert.....	10
L-02: Missing ERC165 support could break ERC1155 Receiver compatibility check.....	11
Informational Severity Issues.....	13
I-01: Permission rules may validate bytes outside the actual target calldata for truncated/custom calldata.....	13
I-02: EIP-7702 path should acknowledge stale data in the imageHash storage slot.....	15
Disclaimer.....	16
About Certora.....	16

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Polygon Trails, Wallet	https://github.com/Oxsequence/wallet-contracts-v3/tree/7702 https://github.com/Oxsequence/trails-contracts	Wallet-v3: b9fb9c7 Trails: 3611e8a3	Solidity

Project Overview

This document describes the security review of **Polygon-Trails-Wallet**. The work was undertaken from **April 29th, 2026** to **May 27th, 2026**.

The following contract list is included in our scope:

```
/wallet-contracts-v3/src/*  
/trails-contracts/src/*
```

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

The Sequence system is a modular smart-wallet framework where authorization is defined by an imageHash with a specific configuration of a signer tree, threshold/weight rules, checkpointing and optional sapient signer modules. Wallets execute signed payloads through calls, self-calls, ERC-4337 UserOperations, EIP-7702 delegated accounts, sessions, recovery, hooks, and static signatures. The wallet-contracts-v3 repo provides the core wallet execution and authentication layer. The trails repo adds intent-oriented utilities such as HydrateProxy, MalleableSapient, RequireUtils, and TimedRefundSapient for runtime value injection, partial calldata commitments, precondition checks, and recovery/refund flows.



These components support flexible wallet control & configuration, delegated app sessions, account-abstraction flows, recovery mechanisms, and intent-style execution, in which some values can be resolved or provided at runtime.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

The Sequence wallet security model depends on correct `imageHash` construction, reliable signer-tree recovery, sound threshold and weight configuration, nonce handling, sapient signer validation, and the integrity of the execution paths that route signed payloads into wallet calls. Our review focused on how these mechanisms interact across configuration changes, sessions, recovery, ERC-4337 and 7702 flows, hooks, static signatures, and delegated execution.

The wallet is intentionally highly flexible, therefore the off-chain wallet configuration is security-critical. If a wallet configures a weak signer tree, overly permissive sapient signer, broad session permission, unsafe hook, or dangerous delegated implementation, the contracts generally treat that as intentional authority. The SDKs are an important practical safety layer and should prevent malformed configurations.

Full wallet threshold signers are treated as fully trusted once threshold is met. They can authorize arbitrary payloads, update configuration, update implementation, install hooks, set static signatures, and generally exercise full wallet authority. Sapient signers are trusted according to the semantics of their implementation, because the parent wallet only verifies the sapient-returned image hash.

Another focus of the review was whether limited-authority paths can exceed their intended boundaries, especially session signers, recovery flows, unsigned session witness fields such as permission or attestation indices, relayer-controlled timing and execution conditions, and sapient module validation logic. We also reviewed non-standard external contract semantics, optional domain separation, cross-chain replay surfaces, and configurations that can turn flexibility into unintended authority.

Sessions And Replay

For sessions, the key assumption is that session signers are constrained by explicit or implicit permissions. The effective authority of a session signer is the union of its configured permissions, and ambiguous permission overlap can affect which usage bucket is charged.

We focused on whether a malicious session signer can escape the permission envelope, cause incorrect usage accounting, exploit call failure semantics, or rely on differences between off-chain assumptions and on-chain validation.

Replay protection is primarily provided by nonce consumption in the selected nonce space. ERC-4337 flows rely on the `EntryPoint/UserOperation` nonce. If chain-agnostic signing is enabled through `payload.noChainId`, the same signed payload may remain valid across chains where the nonce is unused and the relevant configuration exists.

Important SDK and on-chain alignment points include correct construction of session accounting data and permission builders ensuring masks do not rely on bytes outside actual calldata.

Execution Assumptions

The wallet uses low-level calls and generally treats EVM call success as call success. This makes external contract semantics important. Non-standard ERC20s that return false, custom routers with unusual fallback parsing, paused tokens, or calls using ignore/abort behavior can produce outcomes where accounting or authorization succeeds while the intended external effect does not occur.

Hydration Proxy

For hydration and malleability flows, every security-sensitive field modified at runtime must either be committed by the signer/sapient image or intentionally left mutable. `HydrateProxy` and `MalleableSapient` are flexible by design, so their safety depends on a clear boundary between committed fields, hydrated fields, and trusted runtime sources.

Hydrated `delegatecall` targets are especially sensitive because they can execute code in the wallet or proxy context. Any transient executor or proxy that can hold balances, approvals, or privileged roles must also be treated as sensitive unless product flows guarantee it is never left with standing authority.

Areas Reviewed

Areas reviewed included signer-tree recovery, replay and domain separation, static-signature persistence, session permission enforcement, usage-limit accounting, ERC-4337 and 7702 execution-path differences, hook and `delegatecall` privilege boundaries, non-standard token behaviour, hydration-source integrity, and DoS conditions in recovery, simulation, or emergency execution paths.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	-	-	-
Low	2	2	-
Informational	2	2	-
Total	4	4	-

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
Likelihood				

Detailed Findings

ID	Title	Severity	Status
L-01	TimedRefundSapient enforces only .transfer calls, which may silently ignore failed transfers for tokens that don't revert	Low	Acknowledged
L-02	Missing ERC165 support could break ERC1155 Receiver compatibility check	Low	Acknowledged
I-01	Permission rules may validate bytes outside the actual target calldata for truncated/custom calldata	Informational	Acknowledged
I-02	EIP-7702 path should acknowledge stale data in the imageHash storage slot	Informational	Acknowledged

Low Severity Issues

L-01: TimedRefundSapient enforces only .transfer calls, which may silently ignore failed transfers for tokens that don't revert

Severity: Low	Impact: Medium	Likelihood: Low
Files: src/autoRecovery/TimedRefundSapient.sol	Status: Acknowledged	

Description:

There are tokens like [EURS](#) that lack revert behavior and only return false/true, such scenarios are usually handled by the [SafeERC20](#) lib. However in the [TimedRefundSapient](#) we enforce that only direct transfers via .transfer for the [ERC20_TRANSFER_SELECTOR](#) selector can be used.

A possible transfer failure can occur if the token is paused or token-sided logic causes it to fail. In such a scenario, the operation may silently ignore the failure, as the funds would not be transferred and would remain in the wallet. This case is rare, as it requires an external precondition and is recoverable because the funds are still there.

Recommendations:

Consider adding an additional check to ensure the balance was moved.

Customer Response:

Acknowledged.

L-02: Missing ERC165 support could break ERC1155 Receiver compatibility check

Severity: Low	Impact: Medium	Likelihood: Low
Files: Hooks.sol	Status: Acknowledged	

Description:

`Hooks.sol` implements ERC1155 receiver callbacks but misses ERC165 `supportsInterface`.

EIP-1155 says receiver contracts **MUST** implement ERC165:

"Smart contracts implementing the `ERC1155TokenReceiver` interface MUST implement the `ERC-165 supportsInterface` function and signify support for the `ERC1155TokenReceiver` interface to accept transfers. See "ERC1155TokenReceiver ERC-165 rules" for further detail."

Link: <https://eips.ethereum.org/EIPS/eip-1155#erc-1155-token-receiver>

The wallet is not ERC1155 receiver ERC165-compliant with the EIP, thus stricter ERC1155 tokens / or integrations (for example, routers doing pre-checks) may reject or fail to recognize the wallet as a valid ERC1155 receiver.

Example:

<https://github.com/superform-xyz/superform-core/blob/0be774040de50319106dd0cd1a40b348c89f6a64/src/BaseRouterImplementation.sol#L1178-L1185>

Recommendations:

Consider adding:

```
function supportsInterface(bytes4 interfaceId) external pure returns (bool) {
    return interfaceId == 0x01ffc9a7 // ERC165
        || interfaceId == 0x4e2312e0; // ERC1155TokenReceiver
}
```

Customer Response:

Acknowledged. Wallets will use custom hooks to add the `supportsInterface` function itself to support flexibility.

Informational Severity Issues

I-01: Permission rules may validate bytes outside the actual target calldata for truncated/custom calldata

Files:

src/extension/explicit/PermissionValidator.sol

Description:

`PermissionValidator.validatePermission()` extracts rule values from `call.data` using:

```
(bytes32 value,) = call.data.readBytes32(rule.offset);  
value = value & rule.mask;
```

`readBytes32()` does not require the full 32-byte read to be contained inside the actual `call.data` slice. If a rule reads near the end of `call.data`, the read can include bytes outside the logical calldata slice that will be sent to the target before `rule.mask` is applied.

This is usually harmless for standard ABI-encoded Solidity functions, since truncated calldata reverts during ABI decoding. The more relevant case is integrations with custom routers, hooks, fallback handlers that intentionally accept short/non-standard calldata and apply fallback/default behavior.

For example, a non-standard/weird router could use a “fallback” handler that defaults to “withdraw full balance” if the amount field is missing. A session permission may intend to restrict withdrawals by checking ``amount <= 100`` at a fixed offset. A session signer could submit truncated calldata in which the amount is not present. The validator may still read a synthetic value outside the target calldata slice and pass the rule, while the target contract receives different calldata and applies its own default behavior.

The session signer signs the full payload hash, including `call.data` and malformed calldata can make the validator and the target contract reason about different bytes.

Cases where this would be impactful would be rare and would depend on how the contract being called is implemented.

Recommendations:

For custom integrations, especially fallback/custom calldata targets, the permission masks must not rely on overflow bytes

Customer Response:

Acknowledged. That scenario would be incredibly rare. Permissions shouldn't be used when they can't fully validate the payload. This is a misconfiguration. There are many ways an incorrectly configured permission could, in theory, be exploited.

I-02: EIP-7702 path should acknowledge stale data in the imageHash storage slot

Description:

`Stage7702Auth.imageHash()` uses the stored `IMAGE_HASH_KEY` when it is nonzero, and only derives the initial 7702 1-of-1 self-signer config when the slot is empty.

For EIP-7702 accounts, the authority account can change delegated code while keeping storage. If the account previously used another delegated implementation that wrote to the same slot, Sequence may treat that value as an existing config. This case should be acknowledged, as the slot may have been dirty before the first 7702 setup.

Customer Response:

Acknowledged. If `IMAGE_HASH_KEY` is already set, we treat it as an intentional Sequence configuration. When changing delegated implementations, the user must confirm the slot matches the intended config or clear it before delegating again.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.

Finally, Certora delivers ongoing monitoring and incident response, enabling continuous threat detection and prevention, alert triage, and coordinated response to active incidents.